

PHILOSOPHIÆ
NATURALIS
PRINCIPIA
MATHEMATICA.

Autore *ſ* S. NEWTON, *Trin. Coll. Cantab. Soc.* Matheſeos
Profefſore *Lucaſiano*, & Societatis Regalis Sodali.

IMPRIMATUR.
S. PEPYS, *Reg. Soc.* PRÆSES.
Julii 5. 1686.

LONDINI,

Juſſu Societatis Regiæ ac Typis Joſephi Streater. Proſtat apud
plures Bibliopolas. Anno MDCLXXXVII.

INDEX

• Extended Euclidean	3
• Linear Diophantine Equation	4
• Congruence	7
• Linear modular equation	7
• Modular Inverse	8
• Chinese Remainder Theorem	10
• DLP	13
• Euler's Totient Function	14
• Divisors(num-sum-mul)	17
• Factorizations	20
• Mobius Function	21
• Binary_Exponentiation	25
• Formulas	27
• Counting + Probability	30

Extended Euclidean

Notes:

$$a \cdot x + b \cdot y = \gcd(a, b)$$

x & y are the solution

to get more solutions

$$\text{donate } g = \gcd(a, b)$$

$$a \cdot (x + b) + b \cdot (y - a) = g \rightarrow \text{we just added } (ab - ba)$$

$$a \cdot \left(x + \frac{b}{g}\right) + b \cdot \left(y - \frac{a}{g}\right) = g \rightarrow \text{we just added } \left(\frac{ab}{g} - \frac{ba}{g}\right)$$

$$a \cdot \left(x + k \cdot \frac{b}{g}\right) + b \cdot \left(y - k \cdot \frac{a}{g}\right) = g$$

so to compute more solution after having x, y

$$(x_i, y_i) = \left(x + k \cdot \frac{b}{g}, y - k \cdot \frac{a}{g}\right) \rightarrow \text{Bézout's identity}$$

among these pairs only two satisfy that:

$$|x| < \left|\frac{b}{g}\right| \quad \& \quad |y| < \left|\frac{a}{g}\right|$$

and one of them is the solution the EEA gives

Algorithm:

```
11 gcd(11 a , 11 b , 11 &x , 11 &y){ // O(log(min(a , b)))
    if(!b){
        x = 1;
        y = 0;
        return a;
    }
    11 x1 , y1;
    11 g = gcd(b , a % b , x1 , y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
```

Linear Diophantine Equation

Notes:

$$a \cdot x + b \cdot y = c$$

has solution IFF $c \% \gcd(a, b) = 0$

assume we have $\Rightarrow a \cdot x + b \cdot y = g$ where $g = \gcd(a, b)$

this is solvable by EEA and after getting the x, y

$$\text{let } t = \frac{c}{g}$$

so the answer for the original equation is:

$$a \cdot x \cdot t + b \cdot x \cdot t = g \cdot t$$

$$X = x \cdot t, Y = y \cdot t$$

using the same way to get other solutions with Bézout's identity

Algorithm:

```

bool find_any_solution(ll a , ll b , ll c , ll &x0 , ll &y0 , ll &g){ // O(log(min(a , b)))
    if(a == 0 && b == 0 && c != 0) return false;
    if(a == 0 && b == 0 && c == 0) return true;
    g = gcd(abs(a) , abs(b) , x0 , y0);
    if(c % g){
        return false;
    }
    x0 *= c / g;
    y0 *= c / g;
    if(a < 0) x0 = -x0;
    if(b < 0) y0 = -y0;
    return true;
}

```

Notes:

Finding the number of solutions and the solutions in a given interval

$$x \in [\min_x, \max_x] \ \& \ y \in [\min_y, \max_y]$$

1. let $lx_1 = x$ where $x \geq \min_x$, $rx_1 = x$ where $x \leq \max_x$

2. let $lx_2 = x$ where $y \geq \min_y$, $rx_2 = x$ where $y \leq \max_y$

3. the solution:

$$[lx_1, rx_1] \cap [lx_2, rx_2]$$

Algorithm:

```
void shift_solution(ll &x , ll &y , ll a , ll b , ll cnt){ // O(1)
    x += cnt * b;
    y -= cnt * a;
}
ll find_all_solutions(ll a , ll b , ll c , ll minx, ll maxx, ll miny, ll maxy){
//O(log(min(a , b)))
    ll x , y , g;
    if(!find_any_solution(a , b , c , x , y , g)){
        return 0;
    }
    if(a == 0 && b == 0 && c == 0)
        return (maxx - minx + 1) * (maxy - miny + 1);
    if(b == 0)
        return (maxy - miny + 1) * (x >= minx && x <= maxx);
    if(a == 0)
        return (maxx - minx + 1) * (y >= miny && y <= maxy);
    a /= g;
    b /= g;
    // ax + by = c the same as a/g x + b/g y = c/g
    // but gcd(a/g , b/g) = 1
    ll sign_a = a > 0 ? 1 : -1;
    ll sign_b = b > 0 ? 1 : -1;
    //lx1
    shift_solution(x, y, a, b, (minx - x) / b); // shift back to minx
    if (x < minx)
        shift_solution(x, y, a, b, sign_b); // shift forward if out of bound
    if (x > maxx) // no solution
        return 0;
    ll lx1 = x;
    // rx1
    shift_solution(x, y, a, b, (maxx - x) / b); // shift forward to maxx
    if (x > maxx)
        shift_solution(x, y, a, b, -sign_b); // shift back if out of bound
    ll rx1 = x;
    // lx2
    shift_solution(x, y, a, b, -(miny - y) / a); //shift back to miny
    if (y < miny)
        shift_solution(x, y, a, b, -sign_a); //shift forward if out of bound
    if (y > maxy) // no solution
        return 0;
    ll lx2 = x;
```

```

//rx2
shift_solution(x, y, a, b, -(maxy - y) / a); // shift forward to maxy
if (y > maxy)
    shift_solution(x, y, a, b, sign_a); // shif back if out of bound
ll rx2 = x;

if (lx2 > rx2)
    swap(lx2, rx2);
// get intersection
ll lx = max(lx1, lx2);
ll rx = min(rx1, rx2);

if(lx > rx) // no solution
    return 0;

return (rx - lx) / abs(b) + 1; // number of steps to go from lx to rx is the answer
}

```

Notes:

Find the solution with minimum value of $X + Y$

$$X = x + k \cdot \frac{b}{g}$$

$$Y = y - k \cdot \frac{a}{g}$$

$$X + Y = x + k \cdot \frac{b}{g} + y - k \cdot \frac{a}{g} \Rightarrow x + y + k \cdot \left(\frac{b - a}{g} \right)$$

so to minimize this expresion we have three senarios:

1. if $a < b$ so we need to get the minimum value of k
2. if $a > b$ so we need to get the maximum value of k
3. if $a = b$ so all values of k has the same answer

Congruence

$$a \equiv b \pmod{m} \Rightarrow a = b + k \cdot m$$

$$a \equiv b \pmod{n} \rightarrow a \bmod n = b \bmod n \rightarrow (a - b) \bmod n = 0 \rightarrow (a - b) = k \cdot n$$

$$a \cdot x \equiv a \cdot y \pmod{n} \rightarrow x \equiv y \left(\bmod \frac{n}{\gcd(n, a)} \right)$$

$$a^n \equiv b^n \pmod{m} \text{ if } a \equiv b \pmod{m} \text{ for all } n \geq 1$$

$$(a + b)^p \equiv a^p + b^p \pmod{p} \text{ if } p \text{ is prime}$$

$$\text{if } a \equiv b \pmod{m} \text{ \& } c \equiv d \pmod{m} \text{ then } a \pm c \equiv b \pm d \pmod{m} \text{ \& } a \cdot c \equiv b \cdot d \pmod{m}$$

$$x! \pmod{m} = (x \bmod m)! \cdot (m-1)! \pmod{m}, \text{ if } m \text{ is prime}$$

whilson's theorem

if p is prime

$$(p-1)! \equiv -1 \pmod{p} \Rightarrow (p-1)! \pmod{p} = p-1$$

if $a \bmod m = b \bmod m$ then the maximum value of $m = \text{abs}(a - b)$

$$C_r^n \bmod m = \frac{n!}{r!(n-r)!} \bmod m \quad \text{can be solved like this}$$

$$f(n, p) = \max x \text{ where } p^x \leq n$$

$$\prod_p p^{f(n,p)-f(r,b)-f(n-r,b)} \bmod m$$

Linear modular equation

$$a \cdot x \equiv b \pmod{m}$$

if $\gcd(a, n) = 1$:

$$x \equiv b \cdot a^{-1} \pmod{m} \text{ unique solution}$$

else if $(\gcd(a, n) \mid b)$:

divide the equation with $g \Rightarrow a' \cdot x \equiv b' \pmod{n'}$ and solve as the first case

and for $i \in [0, g-1]$, $x_i = x + i \cdot n'$

Algorithm using EEA:

```
vector<ll> modular_linear_equation(ll a , ll b , ll m){ // O(log(min(a , m)))
    ll x , y;
    ll g = gcd(a , m , x , y);
    if(b % g){
        return {};
    }
    vector<ll>ans;
    x = ((x * b / g) % m + m) % m;
    for(int i = 0; i < g; i++){
        ans.push_back((x + i * (m / g)) % m);
    }
    return ans;
}
```

Modular Inverse

Algorithm using power expo:

```
ll power(ll a , ll b , ll m){ // O(logb)
    ll ans = 1;
    while(b){
        if(b & 1){
            ans = (ans * a) % m;
        }
        a = (a * a) % m;
        b >>= 1;
    }
    return ans;
}

ll inv(ll n , ll m){// if m is prime
    return power(a , m - 2 , m); // phi(m) = m - 1 iff m is prime
}

ll inv(ll n , ll m){// if m is not prime
    return power(a , phi(m) - 1 , m);
}

//modular inverse of all numbers from 1 to n modulo m
// if m is prime
vector<ll> modular_inverse(ll n , ll m){ // O(n)
    vector<ll>inv(n + 1);
    inv[1] = 1;
    for(int i = 2; i <= n; i++){
        inv[i] = (m - (m / i) * inv[m % i] % m) % m;
    }
    return inv;
}

// modular inverse of all factorials modulo m
// if m is prime
vector<ll> factorial_modular_inverse(ll n , ll m){ // O(n)
    vector<ll>inv(n + 1) , fact(n + 1);
    fact[0] = fact[1] = 1;
    for(int i = 2; i <= n; i++){
        fact[i] = (fact[i - 1] * i) % m;
    }
    inv[n] = power(fact[n] , m - 2 , m);
    for(int i = n - 1; i >= 0; i--){
        inv[i] = (inv[i + 1] * (i + 1)) % m;
    }
}
```

```

// modular inverse of an array of numbers modulo m
// if m is prime
vector<ll> modular_inverse_array(vector<ll>&a , ll m){ // O(n)
    int n = a.size();
    vector<ll>pre(n + 1) , suf(n + 2);
    pre[0] = suf[n + 1] = 1;
    for(int i = 1; i <= n; i++){
        pre[i] = (pre[i - 1] * a[i - 1]) % m;
    }
    for(int i = n; i >= 1; i--){
        suf[i] = (suf[i + 1] * a[i - 1]) % m;
    }
    vector<ll>inv(n);
    ll x = power(pre[n] , m - 2 , m);
    for(int i = 0; i < n; i++){
        inv[i] = (pre[i] * suf[i + 2] % m) * x % m;
    }
    return inv;
}

```

```

// nCr % m = n! * (r!)^-1 * ((n - r)!)^-1 % m
// if m is prime
ll nCr_mod_prime(ll n , ll r , ll m){
    return (fact[n] * inv_fact[r] % m) * inv_fact[n - r] % m;
}

```

```

ll add(ll a , ll b , ll m){
    return ((a % m) + (b % m)) % m;
}
ll sub(ll a , ll b , ll m){
    return ((a % m) - (b % m) + m) % m;
}
ll mul(ll a , ll b , ll m){
    return ((a % m) * (b % m)) % m;
}

```

Chinese Remainder Theorem

solving where m is pairwise coprime $\Rightarrow \gcd(m_i, m_j) = 1$ for $\forall(i, j)$

$$\left\{ \begin{array}{l} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \dots \end{array} \right\}$$

solving 2 equations:

$$\left\{ \begin{array}{l} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \end{array} \right\}$$

we need to answer this :

$$x \equiv y \pmod{m_1 \cdot m_2}$$

using EEA:

$$n_1 m_1 \cdot n_2 m_2 = 1 \Rightarrow \left\{ \begin{array}{l} n_1 m_1 \equiv 1 \pmod{m_2} \\ n_2 m_2 \equiv 1 \pmod{m_1} \end{array} \right\} \Rightarrow \left\{ \begin{array}{l} n_1 \equiv m_1^{-1} \pmod{m_2} \\ n_2 \equiv m_2^{-1} \pmod{m_1} \end{array} \right\}$$

get n_1, n_2 using modular inverse

so:

$$y = a_1 n_2 m_2 + a_2 n_1 m_1$$

$$x = a_1 n_2 m_2 + a_2 n_1 m_1 \pmod{m_1 \cdot m_2}$$

we can solve any number of equation by solving them two by two

another way:

$$\text{let: } M_i = \prod_{j \neq i} m_j, \quad M = \prod_i m_i$$

$$N_i = M_i^{-1} \pmod{m_i}$$

$$a \equiv \sum_{i=1}^k a_i \cdot M_i \cdot N_i \pmod{M}$$

Algorithm:

```
ll chinese_remainder_theorem(vector<ll>&a , vector<ll>&m){ // O(n * logm)
// m should be pairwise coprime
    ll M = 1;
    for(int i = 0; i < m.size(); i++){
        M *= m[i];
    }
    ll ans = 0;
    for(int i = 0; i < m.size(); i++){
        ll Mi = M / m[i];
        ll y = power(Mi , phi(m[i]) - 1 , m[i]);
        ans = (ans + a[i] * y % M * Mi) % M;
    }
    return ans;
}
```

```
struct cong{
    unordered_map<ll , vector<pair<ll ,ll>>> cong_per_prime;
    vector<ll> a , m;
    cong(){}
    vector<pair<ll ,ll>> factorize(ll m){
        vector<pair<ll ,ll>> factors;
        while(m > 1){
            ll y = spf[m];
            ll cnt = 0;
            while(m % y == 0) cnt ++ , m /= y;
            if(cnt != 0)
                factors.push_back({y , cnt});
        }
        return factors;
    }
    void add_cong(ll _a , ll _m){
        for(auto & [p , e] : factorize(_m)){
            cong_per_prime[p].push_back({_a % power(p , e) , e});
        }
    }
}
```

```

bool prepare(){
    for(auto & [p , congs] : cong_per_prime){
        sort(all(congs) , [&](pair<ll ,ll>&a , pair<ll ,ll>&b){
            if(a.second == b.second)
                return a.first < b.first;
            return a.second < b.second;
        });
        for(int i = 1; i < congs.size(); i ++){
            ll a = congs[i].first , b = congs[i - 1].first;
            ll e1 = congs[i].second , e2 = congs[i - 1].second;
            if((a - b) % power(p , min(e1 , e2)) != 0) return 0;
        }
        a.push_back(congs.back().first);
        m.push_back(power(p , congs.back().second));
    }
    return 1;
}

pair<ll , ll> solve(){
    if(!prepare()) return {-1 , -1};
    return chinese_remainder_theorem(a , m);
}
};

```

$$a^x \equiv b \pmod{m}$$

$$\text{let } k = \lceil \sqrt{m} \rceil$$

we evaluate $a, a^2, a^3 \dots, a^k$

we evaluate $b \cdot a^{-k}, b \cdot a^{-2k}, b \cdot a^{-3k} \dots, b \cdot a^{-rk}$, where $rk > m$

the solution will be in the form:

$$a^i \equiv b \cdot a^{-j \cdot k} \pmod{m}$$

$$a^{j \cdot k + i} \equiv b \pmod{m}$$

```

11 DLP(ll a, ll b, ll m){
    if (!a && !b || (b == 1)) return 1 - (b == 1);
    ll k = sqrt(m) + 1;
    unordered_map<ll, ll> mp;
    ll ai = power(a, k, m);
    ll val = ai;
    for(ll i = 0; i < k; i++){
        mp[val] = i + 1;
        val = (ai * val) % m;
    }
    ai = 1;
    for(ll i = 0; i < k; i++){
        ll val = (b * ai) % m;
        if(mp.find(val) != mp.end()){
            ll ans = mp[val] * k - i;
            return ans;
        }
        ai = (ai * a) % m;
    }
    return -1;
}

```

Euler's Totient Function

$\phi(x) \Rightarrow$ count of numbers in $[1, x]$ that are coprime with x

$$\phi(x) = \sum_{i=1}^x [\gcd(i, x) = 1]$$

if p is prime

$$\phi(p) = p - 1$$

$$\phi(p^k) = p^k - p^{k-1}$$

$$\phi(a \cdot b) = \phi(a) \cdot \phi(b) \text{ if } \gcd(a, b) = 1$$

in general

$$\phi(a \cdot b) = \phi(a) \cdot \phi(b) \cdot \frac{g}{\phi(g)}, \quad g = \gcd(a, b)$$

$$x = p_1^{e_1} \cdot p_2^{e_2} \cdots p_n^{e_n}$$

$$\phi(x) = \phi(p_1^{e_1}) \cdot \phi(p_2^{e_2}) \cdots \phi(p_n^{e_n})$$

$$\phi(x) = x \cdot \left(1 - \frac{1}{p_1}\right) \cdot \left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_n}\right)$$

$$\sum_{d|n} \phi(d) = n$$

Euler's theorem

$$a^{\phi(m)} \equiv 1 \pmod{m}$$

if m is prime it turns to fermat's little theorem

$$a^{m-1} \equiv 1 \pmod{m}$$

$$a^n \equiv a^{n \bmod \phi(m)} \pmod{m}$$

$$\phi(n!) = \begin{cases} n-1 & n \text{ is prime} \\ n & \text{else} \end{cases} \cdot \phi((n-1)!)$$

Algorithm:

```

11 phi(11 n){ // O(sqrt(n))
    11 result = n;
    for(11 i = 2; i * i <= n; i++){
        if(n % i == 0){
            while(n % i == 0){
                n /= i;
            }
            result -= result / i;
        }
    }
    if(n > 1){
        result -= result / n;
    }
    return result;
}

// phi for range [1,n]
vector<11> phi_range(11 n){ // O(n * loglogn)
    bool prime[n + 1];
    vector<11>phi(n + 1);
    memset(prime , 1 , sizeof(prime));
    for(int i = 0; i < n; i++){
        phi[i] = 1;
    }
    for(int i = 2; i <= n; i++){
        if(prime[i]){
            phi[i] = i - 1;
            for(int j = i + i; j <= n; j += i){
                prime[j] = 0;
                int x = j , pow = 1;
                while(x % i == 0){
                    x /= i;
                    pow *= i;
                }
                phi[j] *= (pow / i * (i - 1));
            }
        }
    }
}

```

```

11 phi_factorial(ll n){
// O(n * sqrt(n))if no sieve is used O(n * loglogn) if sieve is used or O(n)
    ll result = 1;
    for(int i = 2; i <= n; i++){
        if(is_prime(i)){
            result *= i - 1;
        }
        else{
            result *= i;
        }
    }
    return result;
}

```

notes:

number of divisors of n :

$$n = p_1^{a_1} \cdot p_2^{a_2} \cdot p_3^{a_3} \cdot \dots \cdot p_k^{a_k}$$

p^a number of divisors equal to $a + 1$

$$\# \text{ divisors of } n = (a_1 + 1) \cdot (a_2 + 1) \cdot (a_3 + 1) \dots (a_n + 1)$$

sum of divisors of n :

let sum of divisors of n to be the function $SD(n)$

$$n = p_1^{a_1} \cdot p_2^{a_2} \cdot p_3^{a_3} \cdot \dots \cdot p_k^{a_k}$$

$$SD(p^a) = p^0 + p^1 + p^2 + \dots + p^a = \frac{p^{a+1} - 1}{p - 1} \Rightarrow \text{geometric series}$$

$$SD(n) = SD(p_1^{a_1}) \cdot SD(p_2^{a_2}) \cdot \dots \cdot SD(p_k^{a_k})$$

Algorithm:

```
// sum of divisors of a number n
ll sum_of_divisors(vector<pair<ll ,ll>>&factors , ll m){
    ll ans = 1;
    for(auto [p , a] : factors){
        ll x = power(p , a + 1 , m);
        x = (x - 1 + m) % m;
        ll y = power(p - 1 , m - 2 , m);
        ans = (ans * x) % m;
        ans = (ans * y) % m;
    }
    return ans;
}

ll divisors(ll num){// function to calculate number of all divisors from [1, n] in O(sqrt(n))
    ll sum = 0, sq = sqrt(num);
    for (ll i = 1; i <= sq; i++) {
        ll add1 = (num / i - i + 1);
        ll add2 = (num / i) - i;
        sum += add1 + add2;
    }
    return sum;
}
```

sum of divisors from 1 to n:

$$ans = \sum_{i=1}^n SD(i)$$

every number in this range divides $\left\lfloor \frac{n}{i} \right\rfloor$ so the summation will be:

$$ans = \sum_{i=1}^n \left\lfloor \frac{n}{i} \right\rfloor \cdot i$$

to calculate it in $O(\sqrt{n})$

$$partialans = \sum_{i=1}^{\sqrt{n}} \left\lfloor \frac{n}{i} \right\rfloor \cdot i$$

Algorithm:

```
11 sum_of_divisors_range(11 n , 11 m){
    11 inv = power(2 , m - 2 , m);
    function<11(11)> sum = [&](11 x){
        return mul(mul(x , add(x , 1 , m) , m) , inv , m);
    };
    function<11(11 , 11 , 11)> range_sum = [&](11 l , 11 r , 11 m){
        11 x = sum(r) % m;
        11 y = sum(l - 1) % m;
        return sub(x , y , m);
    };
    11 ans=0;
    for(11 i = 1; i * i <= n; i++){
        ans += ((n / i) * i) % m;
        ans %= m;
    }
    11 l = (11) sqrt(n);
    for(11 i = (11)sqrt(n) ; i >= 1 ; i --){
        11 r = (n / i) % m;
        11 sum = range_sum(l + 1 , r , m);
        ans = add(ans , mul(sum , i , m) , m);
        l = r;
    }
    return ans;
}
```

```

11 product_of_divisors(vector<pair<ll ,ll>>&factors , ll m){
    ll ans = 1;
    ll number_of_div = 1;
    for(auto [p , a] : factors){
        ll x = power(p , (a * (a + 1)) / 2 , m);
        ans = power(ans , a + 1 , m) * power(x , number_of_div , m);
        ans %= m;
        number_of_div *= (a + 1);
        number_of_div %= (m - 1);
    }
    return ans;
}

```

Algorithm:

```

struct SieveOfEratosthenes{
    int N;
    bool isPrime[N];
    void sieve(){
        for(int i = 0; i < N; i++){
            isPrime[i] = true;
        }
        isPrime[0] = isPrime[1] = false;
        for(int j = 4; j <= N; j += 2){
            isPrime[j] = false;
        }
        for(int i = 3; i*i <= N; i += 2){
            if(isPrime[i]){
                for(int j = i*i; j <= N; j += i){
                    isPrime[j] = false;
                }
            }
        }
    }
};

struct linearSieve{
    int N;
    vector<int> prime;
    int spf[N];
    void sieve(){
        for(int i = 0; i < N; i++){
            spf[i] = 0;
        }
        for(int i = 2; i < N; i++){
            if(spf[i] == 0){
                spf[i] = i;
                prime.push_back(i);
            }
            for(int j = 0; i * prime[j] < N; j++){
                spf[i * prime[j]] = prime[j];
                if(i % prime[j] == 0)
                    break;
            }
        }
    }
};

```

Mobius Function

n is a square free numbers:

if *n* is not divisible by the square of any number greater than 1

$n \bmod x \neq 0, \forall x = y \cdot y$

so *n* prime factors are all different

$$n = p_1 \cdot p_2 \cdot p_3 \cdots p_k$$

$$\mu(n) = \left\{ \begin{array}{ll} 1 & , \quad n = 1 \text{ or } n \text{ is a square free numbers and even prime factors} \\ 0 & , \quad n \text{ is square free and has odd number of prime factors} \\ -1 & , \quad n \text{ is not square free} \end{array} \right\}$$

Algorithm:

```

11 mobius(ll n){ // O(sqrt(n))
    ll result = 1;
    for(ll i = 2; i * i <= n; i++){
        if(n % i == 0){
            n /= i;
            if(n % i == 0){
                return 0;
            }
            result *= -1;
        }
    }
    if(n > 1){
        result *= -1;
    }
    return result;
}

11 mob[N];
void Mobius() // O(n) mobius for range 1 -> n
{
    for(int i = 1; i < N; i++) {
        if(i == 1) mob[i] = 1;
        else{
            if(spf[i / spf[i]] == spf[i]) mob[i] = 0;
            else
                mob[i] = -1 * mob[i / spf[i]];
        }
    }
}

```

Mobius's important notes and problems:

$$\phi(n) = \sum_{d|n} \mu(d) \cdot \frac{n}{d} \quad (1)$$

$$\sum_{i=1}^n [gcd(i, n) = x] = \phi\left(\frac{n}{x}\right) \quad (2)$$

$$\sum_{d|n} \mu(d) = \begin{cases} 0 & , \quad n > 1 \\ 1 & , \quad n = 1 \end{cases} \quad (3)$$

$$\sum_{i=1}^n [d | i] = \left\lfloor \frac{n}{d} \right\rfloor \quad (4)$$

Problem1:

Find the number of co-prime pairs of integers (x, y) in the range [1, n].

$$ans = \sum_{i=1}^n \sum_{j=1}^n [gcd(i, j) = 1], \quad gcd(i, j) = g$$

$$ans = \sum_{i=1}^n \sum_{j=1}^n \sum_{d|g} \mu(d) \Rightarrow \text{from (3)}$$

$$ans = \sum_{i=1}^n \sum_{j=1}^n \sum_{d=1}^n \mu(d) \cdot [d | g]$$

$$ans = \sum_{i=1}^n \sum_{j=1}^n \sum_{d=1}^n \mu(d) \cdot [d | i] \cdot [d | j], \quad [d | g] = [d | i] \cdot [d | j]$$

$$ans = \sum_{d=1}^n \sum_{i=1}^n \sum_{j=1}^n \mu(d) \cdot [d | i] \cdot [d | j], \quad \text{change the order of summation}$$

$$ans = \sum_{d=1}^n \mu(d) \cdot \sum_{i=1}^n \sum_{j=1}^n [d | i] \cdot [d | j]$$

$$ans = \sum_{d=1}^n \mu(d) \cdot \sum_{i=1}^n [d \mid i] \cdot \sum_{j=1}^n [d \mid j]$$

$$ans = \sum_{d=1}^n \mu(d) \cdot \left\lfloor \frac{n}{d} \right\rfloor \cdot \left\lfloor \frac{n}{d} \right\rfloor, \text{ from (4)}$$

$$ans = \sum_{d=1}^n \mu(d) \cdot \left\lfloor \frac{n}{d} \right\rfloor^2$$

Problem2:

Find out the sum of gcd(i, j) for (i, j) in the range [1, n]:

$$ans = \sum_{i=1}^n \sum_{j=1}^n \gcd(i, j)$$

$$\text{let } k = \gcd(i, j)$$

$$ans = \sum_{k=1}^n k \cdot \sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = k]$$

$$\text{let } i = ak, j = bk \text{ then}$$

$$ans = \sum_{k=1}^n k \cdot \sum_{i=1}^{\left\lfloor \frac{n}{k} \right\rfloor} \sum_{j=1}^{\left\lfloor \frac{n}{k} \right\rfloor} [\gcd(a, b) = 1]$$

this part is the same as the previous problem

$$ans = \sum_{k=1}^n k \cdot \sum_{d=1}^{\left\lfloor \frac{n}{k} \right\rfloor} \mu(d) \cdot \left\lfloor \frac{n}{kd} \right\rfloor^2$$

Problem3:

$$ans = \sum_{i=1}^n \left(\frac{n}{gcd(i, n)} \right)$$

$$ans = \sum_{i=1}^n \sum_{k=1}^n [gcd(i, n) = k] \cdot \frac{n}{k}$$

$$ans = \sum_{k=1}^n [k \mid n] \cdot \frac{n}{k} \cdot \sum_{i=1}^n [gcd(i, n) = k]$$

$$\text{let } i = l \cdot k, g = gcd\left(l, \frac{n}{k}\right)$$

$$ans = \sum_{k=1}^n [k \mid n] \cdot \frac{n}{k} \cdot \sum_{l=1}^{\left\lfloor \frac{n}{k} \right\rfloor} [g = 1]$$

$$ans = \sum_{k=1}^n [k \mid n] \cdot \frac{n}{k} \cdot \sum_{l=1}^{\left\lfloor \frac{n}{k} \right\rfloor} \sum_{d \mid g} \mu(d), \text{ from (3)}$$

$$ans = \sum_{k=1}^n [k \mid n] \cdot \frac{n}{k} \cdot \sum_{l=1}^{\left\lfloor \frac{n}{k} \right\rfloor} \sum_{d=1}^{\left\lfloor \frac{n}{k} \right\rfloor} \mu(d) \cdot [d \mid l] [d \mid \frac{n}{k}]$$

$$ans = \sum_{k=1}^n [k \mid n] \cdot \frac{n}{k} \cdot \sum_{d=1}^{\left\lfloor \frac{n}{k} \right\rfloor} \mu(d) \cdot \left[d \mid \frac{n}{k} \right] \cdot \sum_{l=1}^{\left\lfloor \frac{n}{k} \right\rfloor} [d \mid l]$$

$$ans = \sum_{k=1}^n [k \mid n] \cdot \frac{n}{k} \cdot \sum_{d=1}^{\left\lfloor \frac{n}{k} \right\rfloor} \mu(d) \cdot \left[d \mid \frac{n}{k} \right] \cdot \left\lfloor \frac{n}{kd} \right\rfloor$$

$$ans = \sum_{k \mid n} \frac{n}{k} \cdot \sum_{d \mid \frac{n}{k}} \mu(d) \cdot \left\lfloor \frac{n}{kd} \right\rfloor$$

$$ans = \sum_{k \mid n} k \cdot \sum_{d \mid k} \mu(d) \cdot \left\lfloor \frac{k}{d} \right\rfloor$$

$$ans = \sum_{k \mid n} k \cdot \phi(k), \text{ from (1)}$$

Binary_Exponentiation

```
template <typename T, typename X> struct Binary_Exponentiation{

    Binary_Exponentiation() = default;
    Binary_Exponentiation(T MOD) : MOD(MOD) {}
    //binary exponentiation
    T MOD;
    vector<T> applyPermutation(vector<T>& sequence, vector<T>& permutation){

        vector<T> newSequence(sequence.size());

        for (int i = 0; i < sequence.size(); i++){
            newSequence[i] = sequence[permutation[i]];
        }
        return newSequence;
    }
    vector<T> permute(vector<T>& sequence, vector<T>& permutation, T b){

        while (b > 0){
            if (b & 1){
                sequence = applyPermutation(sequence, permutation);
            }
            permutation = applyPermutation(permutation, permutation);
            b >>= 1;
        }
        return sequence;
    }

    vector<vector<T>> add_matrices(vector<vector<T>>& a, vector<vector<T>> &b, bool domod =
false){
        vector<vector<T>> res(a.size(), vector<T>(a[0].size()));
        for (int i = 0; i < a.size(); i++){
            for (int j = 0; j < a[0].size(); j++){
                res[i][j] = a[i][j] + b[i][j];
                if (domod)
                    res[i][j] %= MOD;
            }
        }
        return res;
    }
}
```

```

    vector<vector<T>> multiplies_matrices(vector<vector<T>> &a, vector<vector<T>>& b, bool domod
= false){
    vector<vector<T>> res(a.size(), vector<T>(b[0].size()));
    for (int i = 0; i < a.size(); i++){
        for (int j = 0; j < b[0].size(); j++){
            for (int k = 0; k < a[0].size(); k++){
                T x = a[i][k] * b[k][j];
                if (domod)
                    x %= MOD;
                res[i][j] += x;
                if (domod)
                    res[i][j] %= MOD;
            }
        }
    }
    return res;
}

vector<vector<T>> exp_matrix(vector<vector<T>>& a, X b, bool domod = false){
    vector<vector<T>> res(a.size(), vector<T>(a[0].size()));
    for (int i = 0; i < a.size(); i++){
        res[i][i] = 1;
    }
    while (b > 0){
        if (b & 1){
            res = multiplies_matrices(res, a, domod);
        }
        a = multiplies_matrices(a, a, domod);
        b >>= 1;
    }
    return res;
}
};

```

Formulas

$$\sum_{i=1}^n i = \frac{n \cdot (n + 1)}{2}$$

$$\sum_{i=1}^n i^2 = \frac{n \cdot (n + 1) \cdot (2 \cdot n + 1)}{6}$$

$$\sum_{i=1}^n i^3 = \left(\frac{n \cdot (n + 1)}{2} \right)^2$$

$$\sum_{i=1}^n i^4 = \frac{n \cdot (n + 1) \cdot (2 \cdot n + 1) \cdot (3 \cdot n^2 + 3n - 1)}{30}$$

$$\sum_{i=1}^n C_i = 2^n$$

arithmetic sequence:

$a_n = a + (n - 1)d$, where a is the first element and d is the common ratio

$sum = \frac{n}{2} [2 \cdot a + (n - 1)d]$, where n is terms number

geometric sequence:

$$a_n = a \cdot r^{(n-1)}$$

$$sum = \frac{a \cdot (r^n - 1)}{r - 1}$$

$$sum = \frac{a}{1 - r} , \text{ IFF } |r| < 1$$

$$\gcd(a, b) = \gcd(a - b, b) = \gcd(a \% b, b) = \gcd(a, b \% a)$$

if $\gcd(a, b) = 1$ then $\gcd(a, b - a) = 1$ and vice versa

fibonacci numbers

$$f(0) = 0, f(1) = 1$$

$$f(n) = f(n - 1) + f(n - 2)$$

$$f(m) \mid f(n) \text{ if } m \mid n$$

$$\begin{aligned} (x + y)^n &= \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k \\ &= \sum_{k=0}^n \binom{n}{k} x^k y^{n-k} \end{aligned}$$

The number of ways to put n identical objects into k labeled boxes is

$$\binom{n + k - 1}{n}$$

The number of ways to put n identical objects into k labeled boxes $x_i > 1$

$$\binom{n - 1}{k - 1}$$

every box should have at most one object

$$\binom{n}{k}$$

```
// Find K consecutive integers such that their sum is N
11 findKConsecutiveIntegers(11 n, 11 k){
    double a = 2.0 * n / k + 1 - k;
    a /= 2.0;
    if(a - (11)a == 0){
        return a;
    }
    return -1;
}
```

Counting and Probability

Counting binary strings of length n with no two adjacent 1's

$$dp[0] = 1, dp[1] = 2, dp[n] = dp[n-1] + dp[n-2]$$

if the n th digit is 0, it may follow any legal sequence of length $(n-1)$

If the n th digit is 1, then the $(n-1)$ th digit must be 0.

These sequences can all be obtained by taking every sequence of length $(n-2)$ and appending a 0.

Inclusion-Exclusion

$$2sets := (A \cup B) = A + B - (A \cap B)$$

$$general := \bigcup_i A_i = \sum_k (-1)^{k+1} \left(\sum_{1 \leq i_1 < \dots < i_k \leq n} (A_{i_1} \cap \dots \cap A_{i_k}) \right)$$

The number of ways to distribute n balls into k different boxes where there is k restrictions:

let X = number of ways without the restriction

let A_i = number of ways without the i restriction

$$answer = X - \bigcup_i^k A_i$$

Number of upper-bound integer sums

$$x_1 + x_2 + \dots + x_n = k$$

$$\text{for } \forall i \leq n : 0 \leq x_i \leq m$$

$$ans = \sum_{i=0}^n (-1)^i \binom{n}{i} \binom{n+k-1-i(m+1)}{n-1}$$

The first term is

$$\binom{n+k-1}{n-1} \text{ Number of } n \text{ non-negative integer sums to } k$$

We need to subtract the solutions where one or multiple $x_i > m$

بصمجات

In any cyclic group of order n , the following facts hold:

- For each divisor d of n , there are exactly $\varphi(d)$ elements of order d (where φ is Euler's totient function).